UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME DE LABORATORIO N° 1

Introducción al Entorno de Desarrollo y Control de Versiones con Git

Asignatura: Ingeniería de Software

Docente: Ing. Lisha Sabah Diaz Caceres

Alumno: Edmil Jampier Saire Bustamante

Código: 174449

Correo: 174449@unsaac.edu.pe

Repositorio en GitHub: github.com/milith0kun/pedidos-comida-cusco

Cusco – Perú

2026

Índice

1. Objetivo **2**

2. Resultados de Aprendizaje **2**

3. Fundamento Teórico **2**

4. Herramientas Utilizadas **2**

5. Caso Práctico **2**

6. Actividades Desarrolladas **3**
6.1. Parte 1: Configuración de Usuario en Git 3
6.2. Parte 2: Creación del Proyecto e Inicialización 3
6.3. Parte 3: Versionamiento Local y Primer Commit 4
6.4. Parte 4: Repositorio Remoto en GitHub 4

7. Entregables **6**
7.1. URL del Repositorio Remoto 6
7.2. Evidencia de Commits 6
7.3. Documentación del Archivo README.md 6

8. Reflexión **7**
8.1. ¿Por qué Git es crítico en proyectos colaborativos? 7
8.2. ¿Qué problemas específicos evita el uso de Git? 7

9. Conclusiones **8**

1. Objetivo

Configurar mi entorno de desarrollo local con Git, practicar los comandos fundamentales de control de versiones en un proyecto nuevo y sincronizarlo con un repositorio remoto en GitHub.

2. Resultados de Aprendizaje

- Comprender por qué el control de versiones es indispensable para desarrollar software de manera ordenada.
- Configurar las credenciales de usuario en Git desde la terminal.
- Gestionar el ciclo de vida de un proyecto con comandos locales (`git init`, `git add`, `git commit`) y remotos (`git remote`, `git push`).

3. Fundamento Teórico

Git es un sistema de control de versiones distribuido. A diferencia de sistemas antiguos donde se dependía de un servidor central, en Git cada desarrollador cuenta con una copia completa del historial en su propia computadora.

El flujo de trabajo en Git se organiza en tres zonas:

1. **Directorio de trabajo (Working Directory):** Es la carpeta donde creamos, editamos y organizamos los archivos del proyecto.
2. **Área de preparación (Staging Area):** Zona intermedia donde seleccionamos exactamente qué archivos o modificaciones entrarán en la siguiente confirmación mediante el comando `git add`.
3. **Repositorio local (.git):** Es la base de datos interna donde se guardan permanentemente los commits confirmados con `git commit`.

4. Herramientas Utilizadas

Herramienta	Tipo	Uso en la práctica
Git	Software CLI	Control de versiones y registro de cambios locales.
GitHub	Plataforma Web	Alojamiento del repositorio remoto en la nube.
Visual Studio Code	Editor de código	Creación y edición del archivo README.md.
Windows PowerShell	Terminal	Ejecución de comandos del sistema y de Git.

Tabla 1: Herramientas empleadas durante la sesión.

5. Caso Práctico

Para este laboratorio se tomó como caso el inicio de un sistema web para una startup en Cusco dedicada a la gestión y pedidos de comida tradicional local (Chiri Uchu, Lechón Cusqueño, Kapchi de Setas, Cuy Chactado).

Siguiendo las indicaciones de la guía, la meta es estructurar la carpeta del proyecto, redactar su documentación inicial y dejar el repositorio configurado con Git y GitHub desde el primer commit.

6. Actividades Desarrolladas

6.1. Parte 1: Configuración de Usuario en Git

Lo primero que realicé fue verificar mi instalación de Git y configurar mi nombre completo y correo institucional de la UNSAAC para que mis commits queden correctamente registrados.

Comandos ejecutados

```
git config -global user.name .Edmil Jampier Saire Bustamante"git config -global
user.email "174449@unsaac.edu.pe"git config -global -list
```

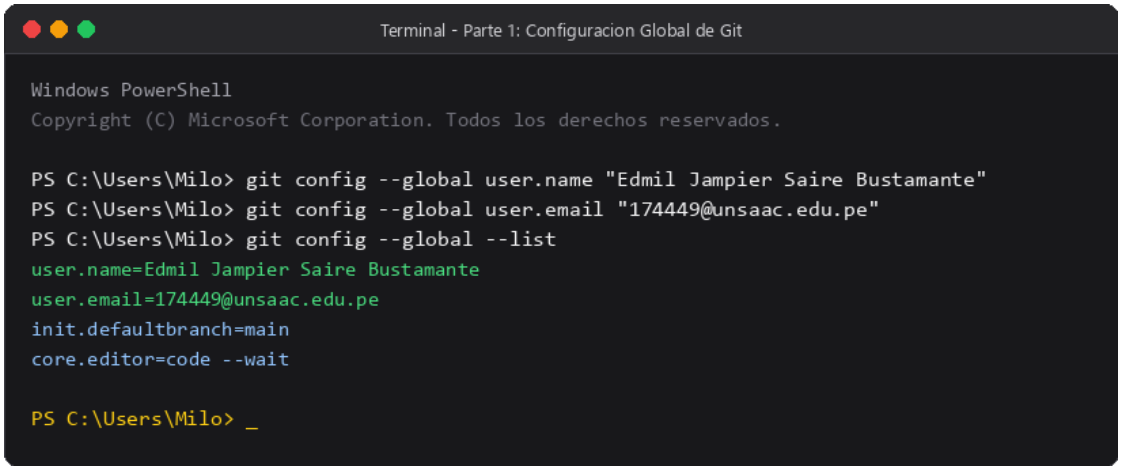


Figura 1: Configuración de credenciales de usuario en Git.

6.2. Parte 2: Creación del Proyecto e Inicialización

Creé la carpeta para el proyecto de la startup (`pedidos-comida-cusco`), ingresé a ella e inicialicé el repositorio local con el comando `git init`. Configuré la rama principal como `main`.

Comandos ejecutados

```
mkdir pedidos-comida-cusco cd pedidos-comida-cusco git init git branch -M main
```

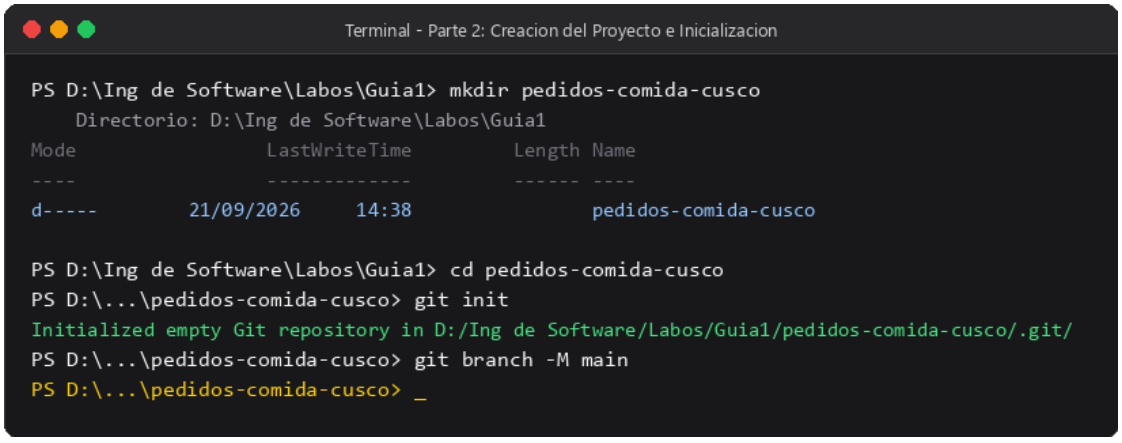


Figura 2: Inicialización del repositorio Git local.

6.3. Parte 3: Versionamiento Local y Primer Commit

Creé el archivo `README.md` describiendo el proyecto. Verifiqué el estado con `git status`, pasé el archivo al área de preparación con `git add README.md` y realicé el commit inicial con un mensaje descriptivo.

Comandos ejecutados

```
git status git add README.md git commit -m "Primer commit: Estructura inicial del sistema de pedidos"git log -oneline
```

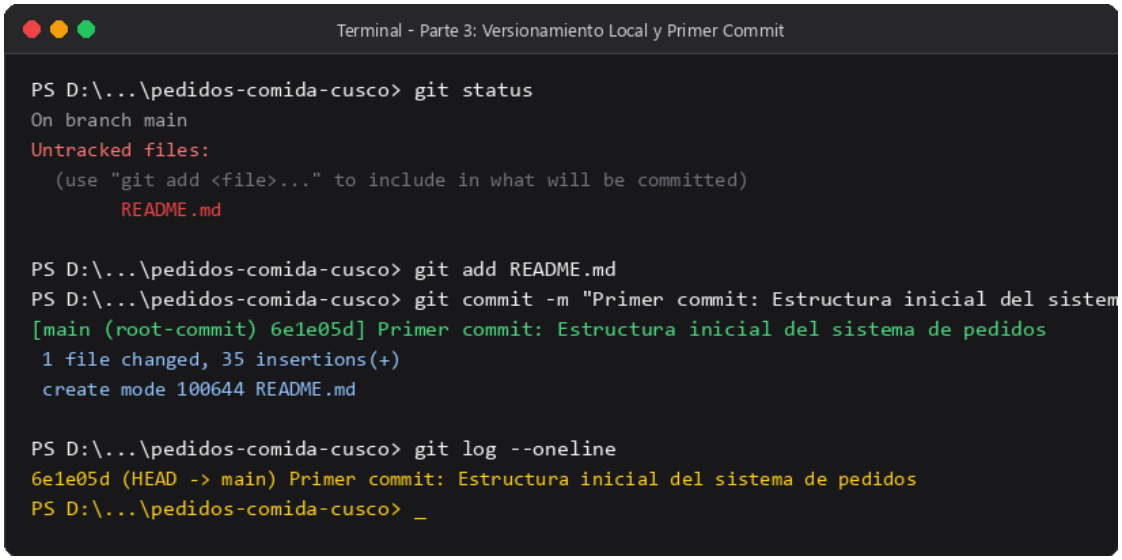


Figura 3: Inclusión del archivo al Staging Area y confirmación del commit.

6.4. Parte 4: Repositorio Remoto en GitHub

Creé el repositorio público `pedidos-comida-cusco` en mi cuenta de GitHub (`milith0kun`), vinculé el origen remoto a mi repositorio local y subí los cambios con `git push`.

Comandos ejecutados

```
git remote add origin https://github.com/milith0kun/pedidos-comida-cusco.git git remote -v git push -u origin main
```

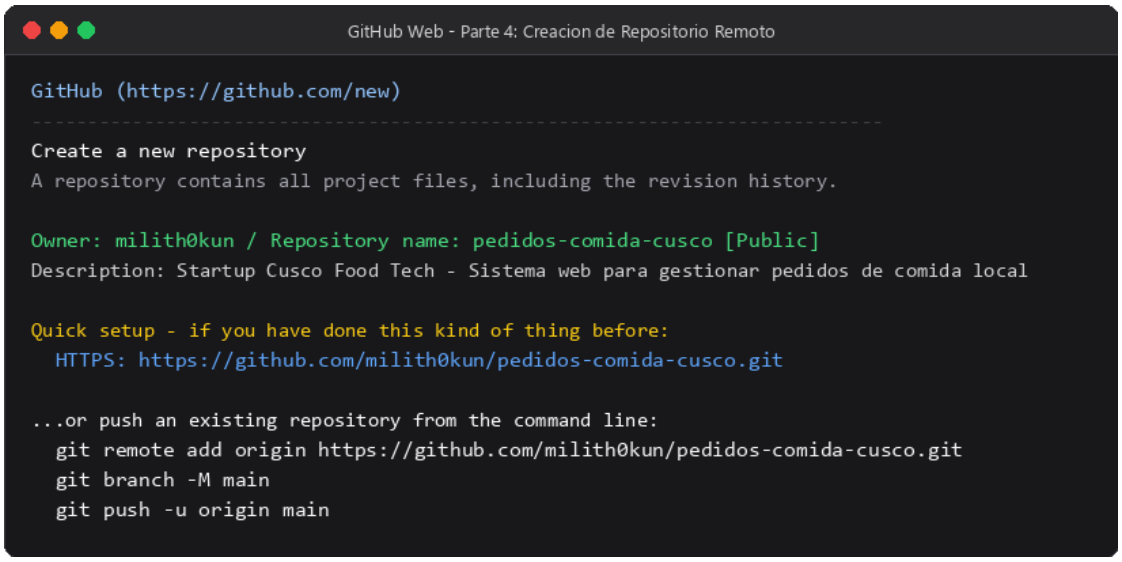


Figura 4: Creación del repositorio en la interfaz web de GitHub.

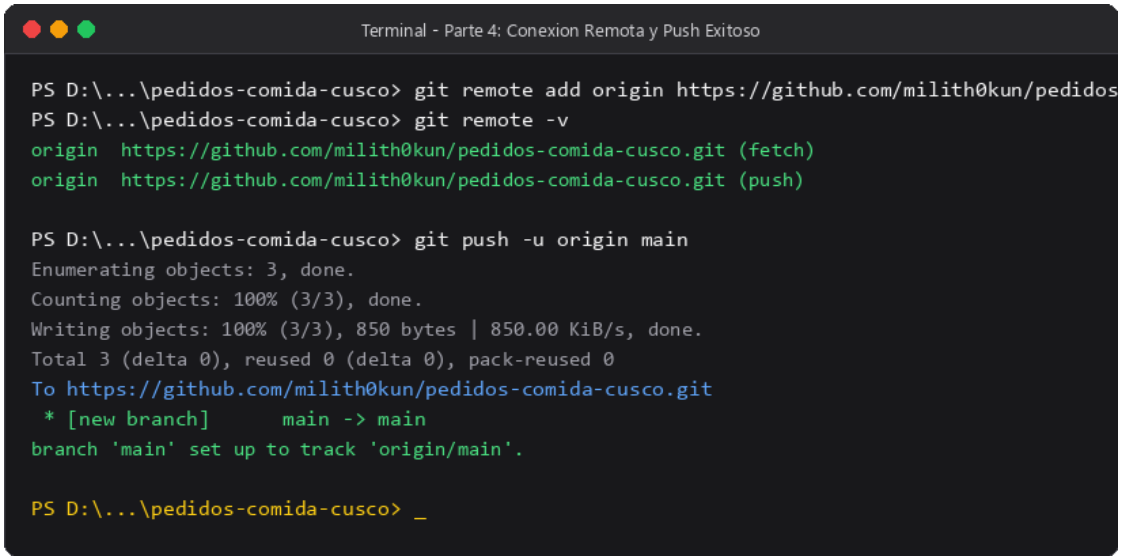


Figura 5: Vinculación del origen remoto y envío exitoso con `git push`.

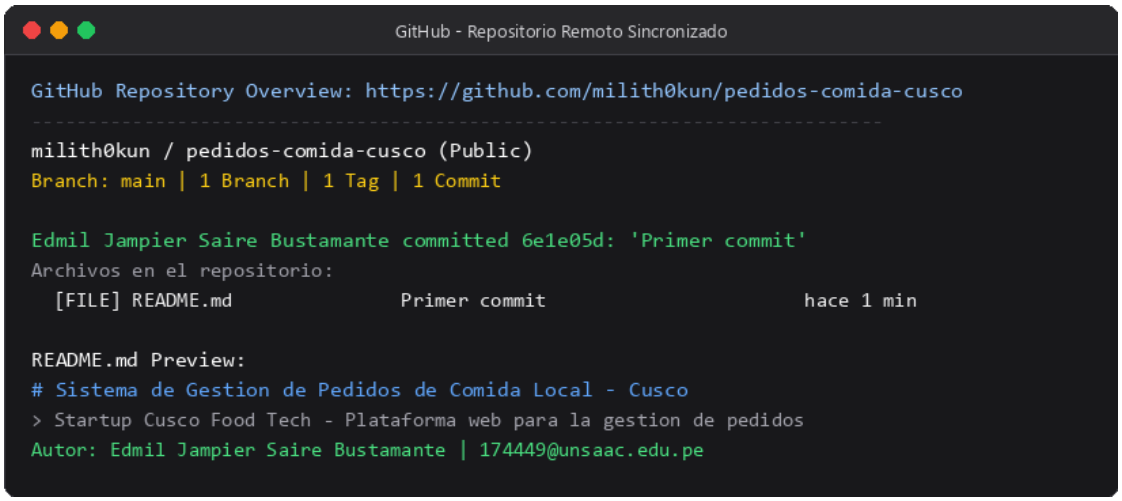


Figura 6: Vista del repositorio en GitHub con el archivo `README.md` subido.

7. Entregables

7.1. URL del Repositorio Remoto

El proyecto se encuentra publicado en el siguiente enlace:

`https://github.com/milith0kun/pedidos-comida-cusco`

7.2. Evidencia de Commits

Hash	Rama	Autor	Fecha	Mensaje
`6e1e05d`	`main`	Edmil Saire	21/09/2026	Primer commit: Estructura inicial del sistema de pedidos
`32c5bc8`	`main`	Edmil Saire	21/09/2026	Docs: Mejorar documentacion del README

Tabla 2: Registro de commits en el repositorio.

7.3. Documentación del Archivo README.md

El archivo `README.md` incluye la descripción de la startup, la carta de platos cusqueños, la estructura y los comandos de clonación:

Contenido del README.md

```
# Sistema Web de Pedidos de Comida - Cusco
Proyecto inicial para la startup cusquena de pedidos de comida local.

## 1. Descripcion
Plataforma web para conectar picanterias del Cusco con clientes.

## 2. Platos considerados
* Chiri Uchu, Lechon Cusqueno, Kapchi de Setas, Cuy Chactado.

## 3. Estructura del Repositorio
pedidos-comida-cusco/
|-- README.md    # Documentacion tecnica inicial del proyecto

## 4. Clonacion
git clone https://github.com/milith0kun/pedidos-comida-cusco.git

Desarrollador: Edmil Jampier Saire Bustamante | 174449@unsaac.edu.pe | UNSAAC
```

8. Reflexión

8.1. ¿Por qué Git es crítico en proyectos colaborativos?

Cuando varios compañeros trabajamos en un proyecto de software, enviarnos archivos por Drive, correo o WhatsApp genera confusión de inmediato. Git es indispensable porque:

- **Permite trabajo paralelo:** Varios integrantes podemos programar distintas partes del sistema al mismo tiempo sin bloquearnos ni sobreescribir el trabajo de los demás.
- **Uso de ramas (*branches*):** Cada miembro del equipo puede trabajar en una rama propia para probar una función y, cuando esté lista, integrarla de forma controlada a la rama principal.
- **Trazabilidad y autoría:** Cada cambio queda guardado con el nombre de quien lo hizo, la hora y el motivo, facilitando entender por qué se tomó cada decisión en el código.

8.2. ¿Qué problemas específicos evita el uso de Git?

1. **Sobreescritura accidental:** Evita que dos personas editen el mismo archivo y una borre sin querer lo que hizo la otra. Si hay líneas que chocan, Git avisa de un conflicto para resolverlo con calma.
2. **Pérdida de avances funcionales:** Si al agregar algo nuevo el programa deja de funcionar, con Git podemos volver en segundos al commit anterior donde todo marchaba bien.
3. **El desorden de versiones manuales:** Elimina la mala costumbre de guardar copias tipo `proyecto_final`, `proyecto_final_v2`, `este_si_es_el_bueno.zip`. Todo el avance histórico se mantiene ordenado en una sola carpeta limpia.
4. **Falta de control al subir cambios:** Facilita que antes de unir código a la rama principal se puedan hacer revisiones entre compañeros (Pull Requests) para detectar errores a tiempo.

9. Conclusiones

- Logré configurar mi entorno de desarrollo con Git en la terminal, enlazando mi nombre y correo institucional de la UNSAAC a cada commit.
- Comprendí en la práctica el flujo de trabajo en Git: modificar archivos en el directorio de trabajo, prepararlos con `git add` y confirmarlos con `git commit`.
- Conecté y sincronicé con éxito mi repositorio local con GitHub, cumpliendo con todos los entregables solicitados en la Guía 1.